

《软件工程理论基础》课程实验三

《软件工程理论基础》课程实验三

一、实验说明

1. 代码提交
2. 实验记录
3. 实验评分

二、智能工厂系统说明

三、系统功能任务需求说明

场景一：基本加工（Basic Manufacturing Mission）

1. 加工任务循环
2. 收集任务循环

场景二：动态加工（Dynamic Manufacturing Mission）

1. 动态加工任务循环
2. 收集任务循环

四、系统监控容错需求说明

场景一监控容错需求

1. 加工机械臂运行状态监控
2. 传送带等待切换条件监控
3. 加工机械臂加工前低电量监控

场景二监控容错需求

1. 收集机械臂运行状态监控
2. 收集机械臂低电量监控

一、实验说明

本次实验围绕**基于领域特定语言（DSL）构建动态场景下的多组件实时系统**展开。你需要根据DSL的语法以及目标系统（智能工厂系统）的说明，依次完成**基本加工**场景和**动态加工**场景下的系统实现。通过DSL实现系统后，调用DSL解析器（`parser`）即可自动生成可在Webots模拟器中可部署运行的C代码，从而检查你的实现是否符合需求。

本实验手册包含系统及场景描述。完整实验资料下载链接<https://box.nju.edu.cn/d/37307caee82b4d13ba36/>，其中附件1包含Webots平台及系统已有基础代码的说明，附件2为DSL手册，Code为代码文件夹。

本次实验最终需要你提交一个名为 `学号_姓名.zip` 的压缩包，请将压缩包提交至<https://box.nju.edu.cn/u/d/7fe9e02f2aae4a14a375/>，提交截止时间为**2026年7月12日23点59分**。压缩包使用 `unzip` 解压后应形成如下文件结构：

```
学号_姓名/  
├── task1.dsl  
├── task2.dsl  
└── 学号_姓名.pdf
```

提交内容说明及评分说明（重要）

1. 代码提交

`task1.dsl` 和 `task2.dsl` 依次为基本加工和动态加工场景下系统的DSL代码实现，即需要你补充完成的 `Code/BasicManufacturingMission/controllers/task1.dsl` 和 `Code/DynamicManufacturingMission/controllers/task2.dsl` 文件。

注意事项：只需提交你编写的 `.dsl` 文件，不需要提交由解析器生成的C代码。

2. 实验记录

学号_姓名.pdf 为实验记录：请任意挑选一个原子场景简要说明其模式切换逻辑（如模式序列、模式切换条件、切换重置动作）；并记录以下4项时间（单位：分钟）：

1. 通过阅读实验手册以及附件1，了解系统需求、模拟平台、框架代码所用时间；
2. 通过阅读附件2，了解DSL 语法所用时间；
3. 完成场景一 `BasicManufacturingMission` DSL代码编码所用时间；
4. 完成场景二 `DynamicManufacturingMission` DSL代码编码所用时间。

3. 实验评分

1. 功能任务正确性（30分）

提交结果应能够在Webots中正常编译并运行，且完成对应场景的任务目标：

- 场景一中，系统能够由传送带、2台加工机械臂和收集机械臂协同完成基本加工任务，并在收集数量达到 `target_cans` 后结束；
- 场景二中，系统能够根据 `factory_state` 动态选择加工机械臂，并在收集数量达到 `target_cans` 后结束。

2. 任务时序正确性（30分）

提交结果应按照第三节要求，满足系统中多组件协同的时序需求，如传送带移动、机械臂初始化、机械臂加工之间具有明确的先后；

3. 监控容错正确性（30分）

提交结果应按照第四节要求，在指定情形下监控安全需求，并在指定情形下执行相应的容错处理；

4. 实验记录完整性（10分）

请按照你的实际情况客观记录你所选原子场景的模式切换逻辑，并完整填写4项用时。原子场景的难易程度及耗时长短均不影响评分，本项仅考查实验是否由本人完成、记录是否完整。

Tip

请在浏览《实验手册》后，先根据《附件1：平台与框架代码介绍》熟悉Webots平台及代码接口，根据《附件2：DSL 介绍》熟悉DSL语法；再使用DSL开发实现系统，并在后续编程实现中随时查阅手册及附件。

二、智能工厂系统说明

智能工厂产线通常由传送带、加工机械臂、收集机械臂、订单系统等多个异构组件共同组成，通过多组件协同，可完成物料输送、加工、收集与订单响应等典型任务。本实验依次开发实现以下两个场景下的智能工厂系统：

- **基本加工场景**：流水线中，传送带与多个加工机械臂协作完成工件加工，同时收集机械臂持续进行成品收集；
- **动态加工场景**：进一步根据订单系统状态动态调整参与加工的机械臂，实现根据实时工厂状态进行柔性生产。

每个场景下的系统实现内容均包括：

- **功能任务控制逻辑**（根据第三节系统功能任务需求）
- **监控容错处理逻辑**（根据第四节系统监控容错需求）

三、系统功能任务需求说明

本实验目标系统中主要组件类型如下：

- **传送带 (ConveyorBelt)**：负责将工件沿产线向前输送，已实现 `Idle` 与 `Running` 等基础功能模式；
- **加工机械臂 (ProcessArm)**：4轴机械臂（4个关节角度及1个腕部角度可调节，1个机械爪可打开、闭合），负责抓取、调整角度、加工和释放工件，已实现 `Idle`、`Grasping`、`Releasing`、`AdjustingAngle`、`Processing` 等基础功能模式；
- **收集机械臂 (CollectArm)**：4轴机械臂（同上），负责感知并收集加工后的工件，已实现 `Idle`、`Grasping`、`Releasing`、`AdjustingAngle` 等基础功能模式；
- **订单系统 (OrderSystem)**：负责在动态加工场景中更新工厂状态，工厂状态用于调整加工机械臂的选择策略。

实验包含的两个智能工厂场景均围绕“传送带输送—加工机械臂加工—收集机械臂收集”的产线流程展开：系统包含加工与收集两个长期**并行运行**的任务部分，二者各自**循环执行**。两个场景下系统均应满足以下要求：

- 传送带与多个加工机械臂协同完成 **加工任务循环**：传送带分段前进，机械臂完成初始化以及加工操作；
- 收集机械臂完成 **收集任务循环**：根据传感器判断货物是否到达收集位置，在货物到达后完成抓取、转移与释放；
- 加工任务循环与收集任务循环 **并行执行**；
- 两个循环的退出条件一致：当收集机械臂累计成功收集 **5 个货物** 后，退出循环，整个系统任务结束。

📢 Important

下面依次介绍各场景。请关注场景中**任务执行条件**及**组件间协同要求**。组件间的协同（如同步开始执行某任务）需通过**显式的协同机制**实现；传送带输送、机械臂初始化、加工和收集等步骤，应在相关前置动作实际完成/相应条件满足后再启动，不得通过固定延时实现组件协同或假定前置动作已经完成。

实际上，我们在系统基础功能模块中已植入随机故障。因此，除机械爪开/闭动作通过周期计数器控制持续周期、加工动作通过加工时间控制时长外，其他任务步骤均不能依赖固定延时推进，否则无法保证系统稳定运行。

场景一：基本加工 (Basic Manufacturing Mission)

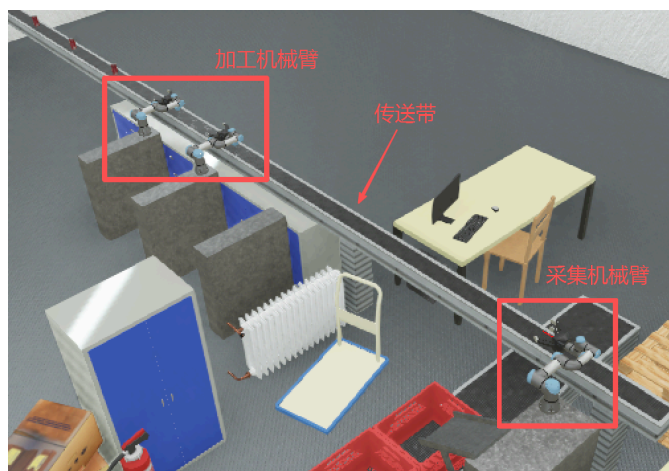
代码路径：

`/Code/BasicManufacturingMission/`

系统组件：

场景一由 1 条传送带、2 台加工机械臂和 1 台收集机械臂组成：

- `belt`：传送带；
- `p_arm1`、`p_arm2`：加工机械臂；
- `c_arm`：收集机械臂。



整体流程：

场景一包含以下两个 **并行执行** 的任务部分：

1. **加工任务循环**：belt 与 p_arm1、p_arm2 协同完成货物加工；
2. **收集任务循环**：c_arm 根据传感器判断货物是否到达，并对到达的货物进行收集。

两个任务部分同时开始，并各自循环执行。c_arm 累计收集数量未达到 **5 个**，则循环继续运行；当收集数量达到 **5 个** 后，不再循环，系统任务完成。

1. 加工任务循环

每一轮加工任务依次执行4个阶段，流程如下：

1. **传送带第一次前进**：传送带向前移动 **0.5 m**，速度为 **0.2m/s**；
2. **加工机械臂初始化**：p_arm1 和 p_arm2 **并行** 调整自身姿态，调整至与传送带平行（方便抓取货物）的位置；
3. **传送带第二次前进**：传送带继续向前移动 **0.5 m**，速度为 **0.2m/s**；
4. **加工机械臂加工**：p_arm1 和 p_arm2 **并行** 对到达加工位置的货物执行加工动作。

其中，每台加工机械臂的一次完整加工阶段依次包括如下动作步骤：

1. 抓取货物；
2. 调整到加工位置；
3. 执行加工，持续 **3.0 s**；
4. 调整回与传送带平行的位置；
5. 释放货物；
6. 调整回加工位置，等待下一轮任务。

执行与协同要求：

- 每轮加工任务按上述 4 个阶段顺序推进：只有当前阶段涉及的所有组件均完成对应动作后，系统才能进入下一阶段。具体来说，传送带完成第一次前进后，2 台加工机械臂才能开始初始化；2 台加工机械臂均完成初始化后，传送带才能开始第二次前进；传送带完成第二次前进后，2 台加工机械臂才能开始加工。
- 在“加工机械臂加工”阶段，p_arm1 和 p_arm2 同时开始，并行执行各自的完整加工动作。**两机械臂不需要在加工阶段的 6 个内部动作步骤之间逐个同步**，即不要求 p_arm1 和 p_arm2 同时调整角度、同时释放等；只需保证二者同时进入加工阶段，且二者均完成完整加工动作步骤后，加工阶段才视为结束。

2. 收集任务循环

不同于加工机械臂空闲时处于加工位置等待，收集机械臂 `c_arm` 空闲时处于与传送带平行位置，以便通过传感器判断货物是否可收集；当检测到货物到达收集位置后，执行一次收集动作，流程如下：

1. 在空闲模式等待时根据传感器判断货物是否到达收集位置；
2. 抓取货物；
3. 调整自身角度至释放位置；
4. 释放货物；
5. 调整自身角度至与传送带平行位置，将累计收集数量加 1；
6. 返回至空闲模式等待。

执行与协同要求：

- 收集机械臂应根据传感器判断货物到达，而不是按照固定时间间隔盲目收集；
- 收集数量是加工及收集不再循环的判断依据。当收集数量达到 5 个后，两个并行任务均不应再执行新一轮循环，两并行任务均完成则系统任务完成。

场景二：动态加工（Dynamic Manufacturing Mission）

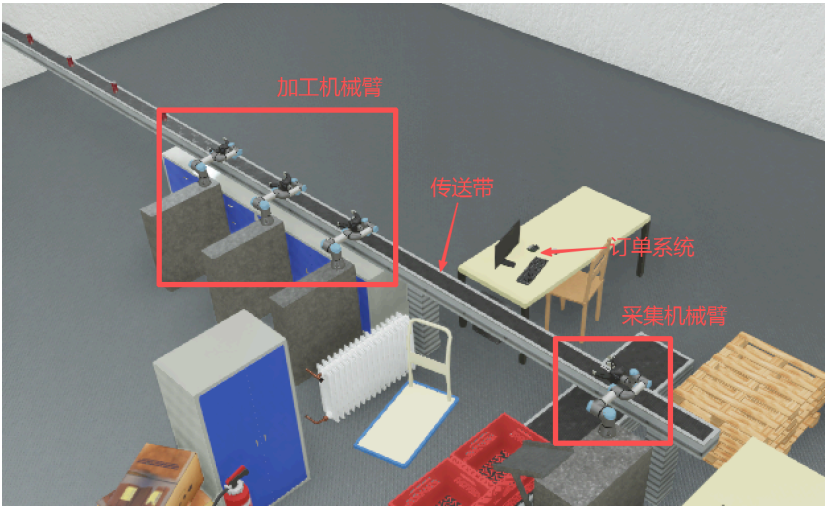
代码路径：

`/Code/DynamicManufacturingMission/`

系统组件：

场景二在场景一基础上增加了 1 台加工机械臂和 1 个订单系统：

- `belt`：传送带；
- `p_arm1`、`p_arm2`、`p_arm3`：加工机械臂；
- `c_arm`：收集机械臂；
- `ordersystem`：订单系统。



整体流程：

场景二同样包含两个**并行执行**的任务部分：

- 1. **动态加工任务循环：** `belt` 根据工厂状态，选择不同的加工机械臂组合完成加工；
- 2. **收集任务循环：** `c_arm` 根据传感器判断货物是否到达，并对到达的货物进行收集。

两个任务部分同时开始，并各自循环执行。当 `c_arm` 累计收集数量达到 **5 个** 后，不再循环，系统任务完成。

1. 动态加工任务循环

动态加工任务循环与场景一的加工任务循环类似，区别在于：场景一中每轮加工始终由固定的 2 台加工机械臂完成；而场景二中，订单系统会自动根据订单更新工厂状态，每轮加工需根据当前**工厂状态**，进行**分支选择**，在多种加工流程分支之间选择其中一种执行。不同分支对应不同的机械臂组合，但每轮均仍遵循“传送带前进—机械臂初始化—传送带前进—机械臂加工”的基本流程。

每轮动态加工任务的分支选择规则如下：

- 1. 如果 `factory_state == 1`，则执行由 `p_arm1` 和 `p_arm2` 参与的双臂加工流程；
- 2. 否则，如果 `factory_state == 2`，则执行由 `p_arm1` 和 `p_arm3` 参与的双臂加工流程；
- 3. 否则，如果 `factory_state == 3`，则执行由 `p_arm2` 和 `p_arm3` 参与的双臂加工流程；
- 4. 否则，执行由 `p_arm1`、`p_arm2` 和 `p_arm3` 共同参与的三臂加工流程。

工厂状态	本轮参与加工的机械臂
<code>factory_state == 1</code>	<code>p_arm1</code> 、 <code>p_arm2</code>
<code>factory_state == 2</code>	<code>p_arm1</code> 、 <code>p_arm3</code>
<code>factory_state == 3</code>	<code>p_arm2</code> 、 <code>p_arm3</code>
其他情况	<code>p_arm1</code> 、 <code>p_arm2</code> 、 <code>p_arm3</code>

其中，双臂加工流程与场景一中的加工任务循环一致，每一轮仍包括：

- 1. **传送带第一次前进：**传送带向前移动 **0.5 m**，速度为 **0.2m/s**；
- 2. **加工机械臂初始化：**被选中的 2 台加工机械臂 **并行** 调整到与传送带平行的位置；
- 3. **传送带第二次前进：**传送带继续向前移动 **0.5 m**，速度为 **0.2m/s**；
- 4. **加工机械臂加工：**被选中的 2 台加工机械臂 **并行** 对到达加工位置的货物执行加工动作。

三臂加工流程与双臂加工流程类似，只是参与加工的机械臂变为 3 台。

其中，在加工阶段，被选中的每台加工机械臂均执行与场景一相同的加工动作步骤：

- 1. 抓取货物；
- 2. 调整到加工位置；
- 3. 执行加工，持续 **3.0 s**；
- 4. 调整回与传送带平行的位置；
- 5. 释放货物；
- 6. 调整回加工位置，等待下一轮任务。

执行与协同要求：

- 每一轮动态加工任务开始时，系统根据当前工厂状态 **进行分支选择**，选择一个完整的加工分支执行。若工厂状

态在本轮加工过程中发生变化，当前轮次不应中途切换到其他分支。

- 被选中的加工分支内部仍按阶段依次顺序推进。在双臂加工分支中，只有被选中的 2 台加工机械臂参与本轮任务；未被选中的加工机械臂应保持空闲等待状态。在三臂加工分支中，3 台加工机械臂均参与本轮任务；只有 3 台机械臂均完成初始化后，传送带才能继续前进；只有 3 台机械臂均完成完整加工阶段后，本轮加工任务才视为结束。
- 在机械臂加工阶段，同样不要求不同机械臂在 6 个内部加工步骤之间逐步同步，即不要求它们同时抓取、同时调整角度或同时释放；只需保证所有参与本轮任务的机械臂同时开始，并且在均完成完整加工动作后，当前加工分支才结束。

2. 收集任务循环

场景二中的收集任务循环与场景一相同，不再赘述。

四、系统监控容错需求说明

为保障安全，智能工厂系统在运行中需实时监控关键安全性质，并在检测到异常时执行对应的容错操作，需求如下。

Important

所有用于监控检测的监控器与用于容错处理的容错模式/容错函数均已实现，系统实现过程中，你只需根据需求，在合适的监控时机调用监控器并绑定容错处理操作。

场景一监控容错需求

1.加工机械臂运行状态监控

监控器： `check_arm_state()`

- **监控场景**：整个BasicManufacturingMission场景。
- **监控对象**：所有加工机械臂（ `ProcessArm` ）。
- **监控时机**：在 `ProcessArm` 的以下模式中 **持续** 监控： `Idle`、 `Grasping`、 `Releasing`、 `AdjustingAngle`、 `Processing`。
- **容错动作**：若检测到机械臂异常，立即切换至 `Restarting` 容错模式。

2.传送带等待切换条件监控

监控器： `check_conveyor_wait()`

- **监控场景**：整个BasicManufacturingMission场景。
- **监控对象**：传送带（ `ConveyorBelt` ）。
- **监控时机**：在 `ConveyorBelt.Running` 模式中 **持续** 监控。
- **容错动作**：若检测到任一机械臂处于故障或重启状态，传送带应切换至 `waiting` 容错模式。

说明：传送带主动监控自身是否需要进入等待模式，以避免在任一机械臂故障期间继续输送工件进而导致加工位置错位或任务状态不一等。

3.加工机械臂加工前低电量监控

监控器： `check_arm_low_battery()`

- **监控场景**：整个BasicManufacturingMission场景。
- **监控对象**：所有加工机械臂（ `ProcessArm` ）。
- **监控时机**：进入 `ProcessArm.Processing` 模式之前。
- **容错动作**：若检测到低电量状态，调用容错函数 `reportLowBattery` 记录或报告低电量信息。

场景二监控容错需求

场景二 保留场景一的所有监控容错需求，并在场景一的监控容错需求基础上，由于在动态加工场景下，采集机械臂的工作压力增大，因此需要 额外对收集机械臂 进行“机械臂运行状态监控”与“机械臂低电量监控”。

1.收集机械臂运行状态监控

监控器： `check_arm_state()`

- **监控场景**：整个DynamicManufacturingMission场景。
- **新增监控对象**：收集机械臂（ `CollectArm` ）。
- **对应监控时机**：在 `CollectArm` 的以下模式中 **持续** 监控： `Idle`、 `Grasping`、 `Releasing`、 `AdjustingAngle`。
- **对应容错动作**：若检测到机械臂异常，立即切换至 `Restarting` 容错模式。

2.收集机械臂低电量监控

监控器： `check_arm_low_battery()`

- **监控场景**：整个DynamicManufacturingMission场景。
- **新增监控对象**：收集机械臂（ `CollectArm` ）。
- **对应监控时机**：退出 `CollectArm.AdjustingAngle` 模式后，即收集机械臂完成角度调整后。
- **对应容错动作**：若检测到低电量状态，调用容错函数 `reportLowBattery` 以记录或报告低电量信息。